

CV中的Transformer [3]—Swin Transformer

导师：电子羊

课程回顾

Previously on Transformer in CV



ViT

一种纯Transformer的图像分类模型，有点在于Global以及Scale能力

Pyramid

图像金字塔结构，被广泛运用于传统及深度学习领域中。

MHA复杂度

Q@KT的操作无论时间还是空间复杂度都为 $O(n^2)$ 这在Token很多时无法承受

PVT

一种运用了金字塔结构的ViT，在底层使用了SRA限制了K的数量，减小了计算量

第一部分：论文介绍

The first lesson: the paper
~~guide~~



Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

Swin Transformer

作者: Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei

Zheng Zhang, Stephen Lin, Baining Guo

单位: MSRA

发表会议及时间: ICCV2021 (Best Paper)

目录

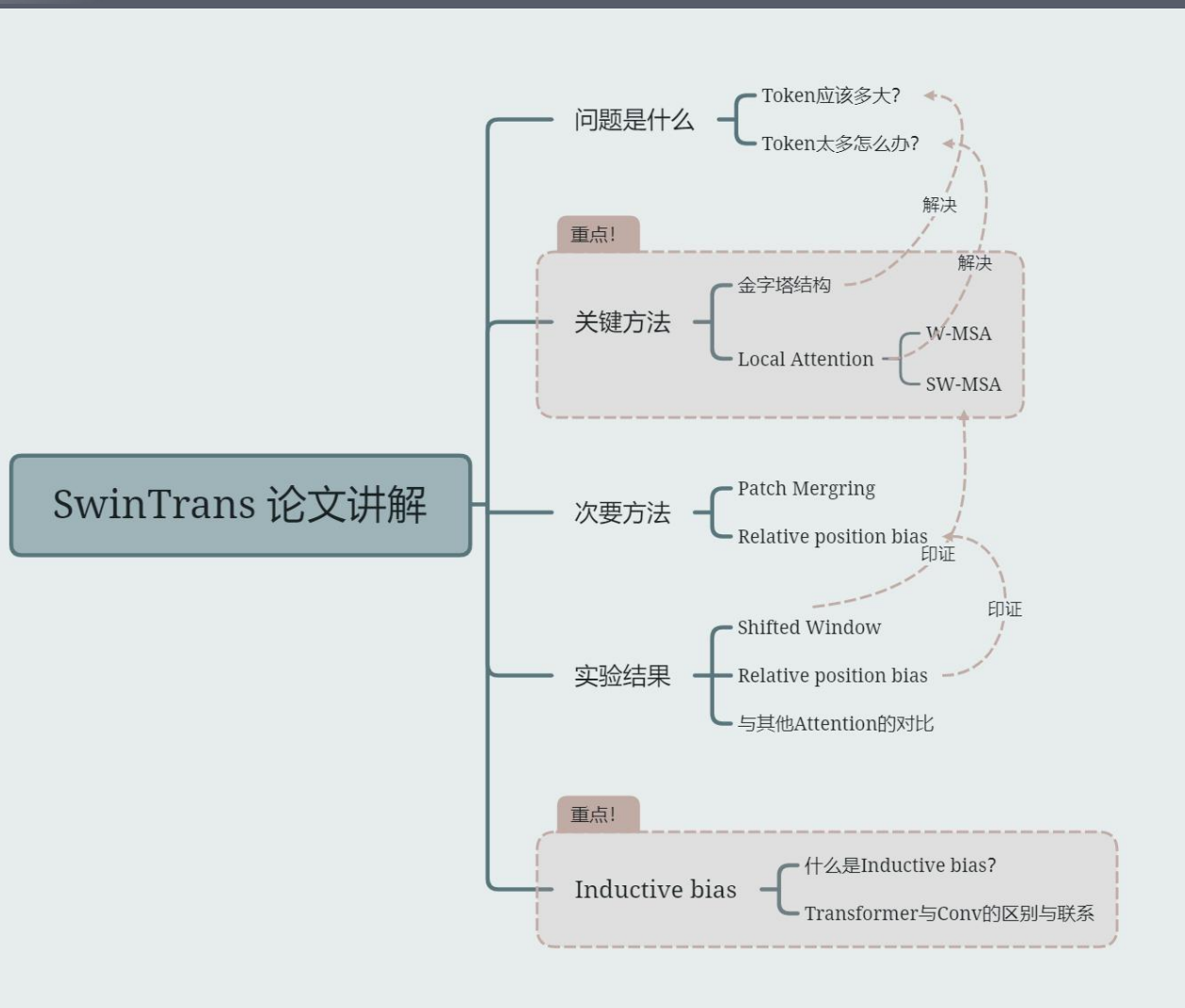
1/ 论文研究背景、成果及意义

2/ 论文泛读

3/ 论文精读

4/ 本课回顾及下节预告

讲解顺序



论文研究背景、成果及意义



研究背景

Research background



重点 重点来了！

Transformer从NLP到CV

ViT出现后，出现了很多Transformer运用到CV各个任务上的Paper

但作为这两个任务有一些根本上的不同，导致一些任务中Transformer不那么好用。

原文给出了两点不同，一个是Token的大小，一个是Token的数量，这两点和上节课所说的显存占用以及表示方法是同一个问题的两种不同表达方式。

研究背景

Research background

与NLP的区别

1. 显存占用 - Token数量

2. 表示方法 - Token应该多大

- Token为最小的处理单元
- 在NLP中有很多符合直觉的Token定义，比如说单词，或者说中文中几个字组成的词。
- 但在CV中，Token应该多大呢？

研究背景

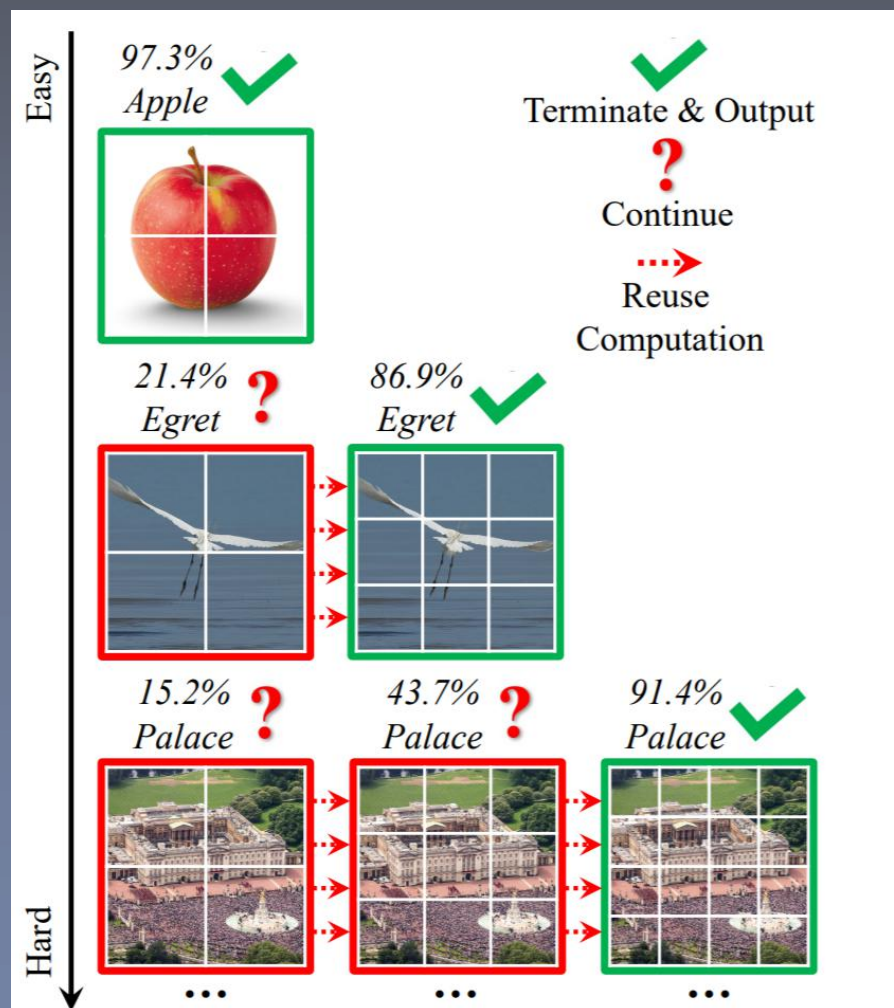
Research background

- 对于不同类别来说，复杂程度不一样

与NLP的区别

1. 表示方法 - Token应该多大

2. 显存占用 - Token数量



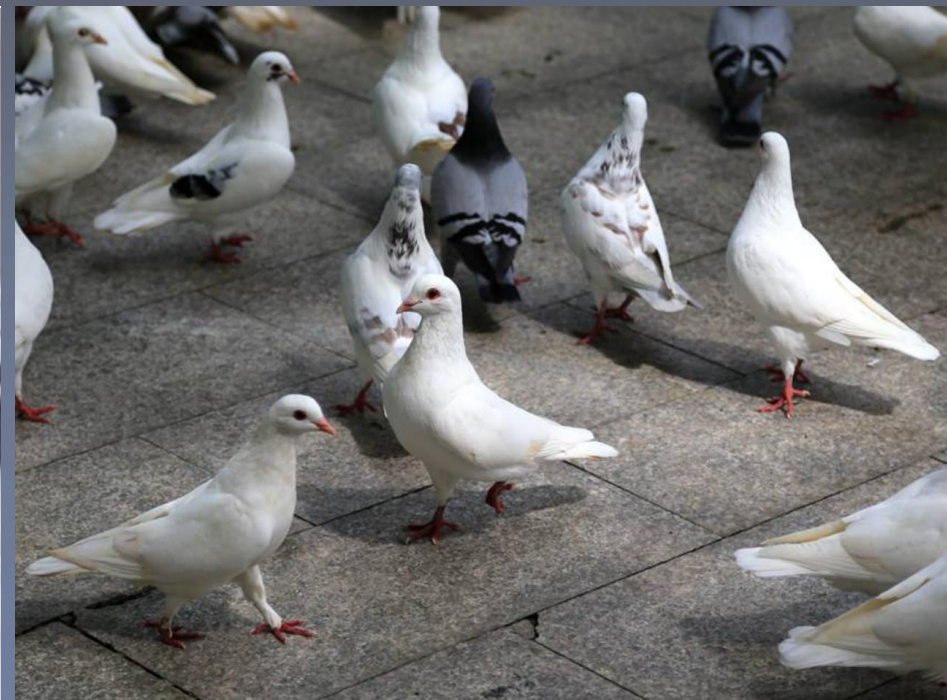
研究背景

Research background

- 对于同一类别来说，大小不一样

与NLP的区别

1. 表示方法 - Token应该多大
2. 显存占用 - Token数量



研究背景

Research background

- 甚至一些不太规则的类别, sky, grass...

与NLP的区别

1. 表示方法 - Token应该多大
2. 显存占用 - Token数量



研究背景

Research background

与NLP的区别

1. 表示方法 - Token应该多大

2. 显存占用 - Token数量

- Token指的是模型运行中**最小的处理单元**。
- NLP中的Token一般指的是一个**单词**，Token的数量就是一段文字中单词的数量，**一般不会特别多**。
- 比如微博内容，**140字**
- 比如高考作文，一般不会超过**1200**

研究背景

Research background

与NLP的区别

1. 表示方法 - Token应该多大

2. 显存占用 - Token数量

- Token指的是模型运行中最小的处理单元。
- CV中的Token指的是一个Patch，Token的数量就是Patch的数量，下游任务中一般会很多。
- 比如224输入，4倍下采样的情况下，Token数量为56x56，3000多个

研究背景

Research background

与NLP的区别

1. 表示方法 - Token应该多大

2. 显存占用 - Token数量

- CV中使用MHA的dilemma
- Patch小 $\rightarrow$ Token数量多 $\rightarrow$ MHA复杂度高 $\rightarrow$ 资源不足
- Patch大 $\rightarrow$ Token数量少 $\rightarrow$ 分辨率低 $\rightarrow$ 下游任务性能差
- 性能 or 速度，这是个严重的问题

研究成果

Research Results

Swin Transformer在多种任务上均取得了不小的进步

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2
(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

(a) Various frameworks									
Method	Backbone	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS		
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0		
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3		
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3		
	Swin-T	47.2	66.5	51.3	36M	215G	22.3		
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6		
	Swin-T	50.0	68.5	54.2	45M	283G	12.0		
Sparse	R-50	44.5	63.4	48.2	106M	166G	21.0		
R-CNN	Swin-T	47.9	67.3	52.3	110M	172G	18.4		
(b) Various backbones w. Cascade Mask R-CNN									
	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅	#param.	FLOPs	FPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G	10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G	18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G	15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G	12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G	12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G	10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G	11.6
(c) System-level Comparison									
Method	AP ^{box}	mini-val AP ^{mask}	test-dev AP ^{box}	test-dev AP ^{mask}	#param.	FLOPs			
RepPointsV2* [11]	-	-	52.1	-	-	-			
GCNet* [6]	51.8	44.7	52.3	45.4	-	1041G			
RelationNet++* [12]	-	-	52.7	-	-	-			
SpineNet-190 [20]	52.6	-	52.8	-	164M	1885G			
ResNeSt-200* [75]	52.5	-	53.3	47.1	-	-			
EfficientDet-D7 [58]	54.4	-	55.1	-	77M	410G			
DetectoRS* [45]	-	-	55.7	48.5	-	-			
YOLOv4 P7* [3]	-	-	55.8	-	-	-			
Copy-paste [25]	55.9	47.2	56.0	47.4	185M	1440G			
X101-64 (HTC++)	52.3	46.0	-	-	155M	1033G			
Swin-B (HTC++)	56.4	49.1	-	-	160M	1043G			
Swin-L (HTC++)	57.1	49.5	57.7	50.2	284M	1470G			
Swin-L (HTC++)*	58.0	50.4	58.7	51.1	284M	-			

Table 2. Results on COCO object detection and instance segmentation. [†]denotes that additional deconvolution layers are used to produce hierarchical feature maps. * indicates multi-scale testing.

ADE20K		val mIoU	test score	#param.	FLOPs	FPS
Method	Backbone					
DANet [22]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [10]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [23]	ResNet-101	45.9	38.5	-	-	-
DNL [68]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [70]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [66]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [70]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [10]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [10]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [78]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Table 3. Results of semantic segmentation on the ADE20K val and test set. [†] indicates additional deconvolution layers are used to produce hierarchical feature maps. [‡] indicates that the model is pre-trained on ImageNet-22K.

研究意义

Research Meaning



深度之眼
deepshare.net



重点 重点来了！

Swin Transformer的研究意义

- 这是第一篇被大家广泛接受的在下游任务中使用纯Transformer结构的方式
- 后续有非常多的工作都在Swin Transformer的基础上做的

研究意义

Research Meaning



深度之眼
deepshare.net



重点 重点来了!

Swin Transformer的研究意义

- 这是第一篇被大家广泛接受的在下游任务中使用纯Transformer结构的方式
- 这篇论文后续有非常多的工作，3月底至今已经500+的引用了。

| Image Classification: Included in this repo. See [get_started.md](#) for a quick start.

| Object Detection and Instance Segmentation: See [Swin Transformer for Object Detection](#).

| Semantic Segmentation: See [Swin Transformer for Semantic Segmentation](#).

| Self-Supervised Learning: See [Transformer-SSL](#).

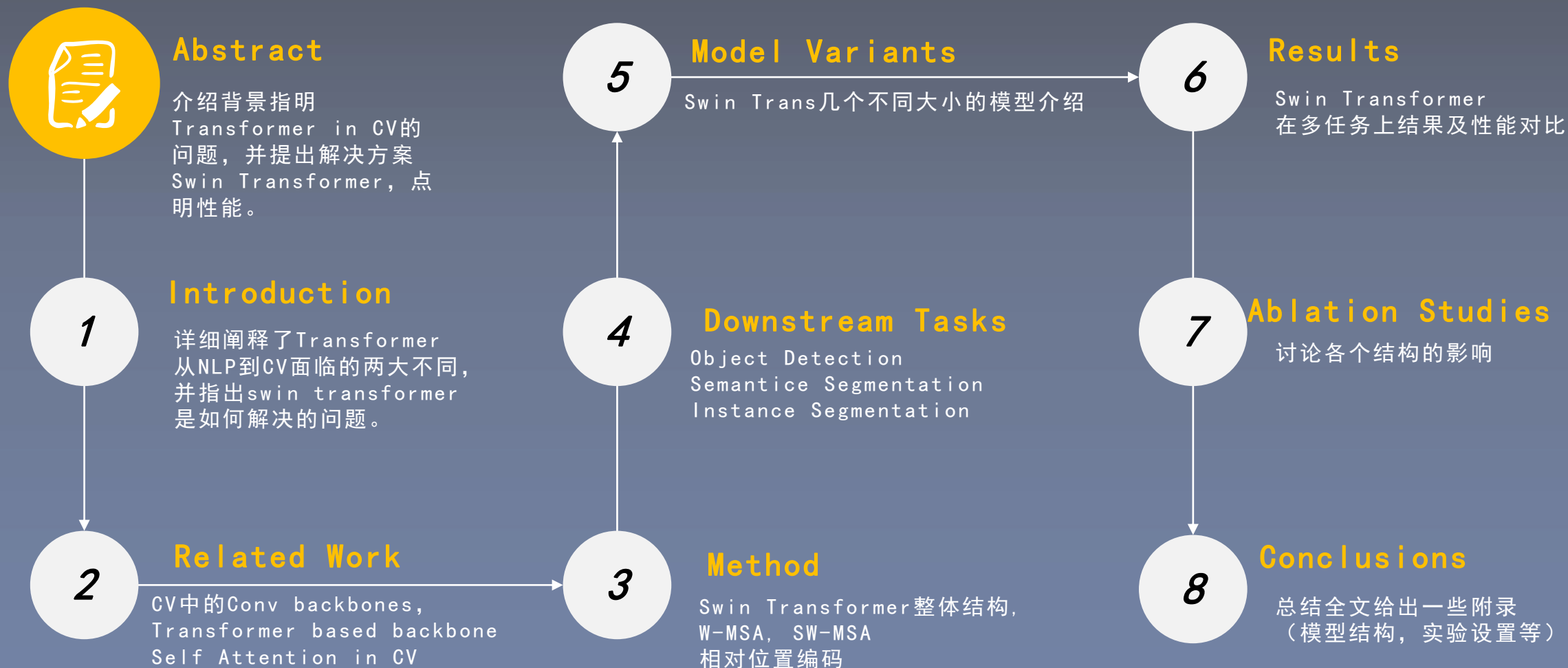
| Video Swin Transformer: See [Video Swin Transformer](#).

论文泛读

Structure of Paper

论文结构

Structure of Papers



第二部分：论文精读

The second lesson: the paper in detail

摘要

abstract

摘要核心

1. 本文提出一种可以适用于多种任务的backbone->swin transformer
2. Transformer迁移到CV中有两点挑战->物体尺度不一，图像分辨率大。
3. 为了解决尺度不一的问题，Swin Transformer使用了分层的结构(Pyramid)
4. 为了能够在高分辨率上运行，Swin Transformer限制了attention的计算范围。
5. Swin Transformer在多种任务上取得了非常好的性能。



目录

1/ 论文算法模型总览

2/ W-MSA

3/ SW-MSA

4/ Inductive bias

5/ Relative position bias

6/ 实验设置及结果

7/ Ablation Studies

8/ 论文总结

论文算法模型总览

Swin Transformer 结构

Architecture of Swin

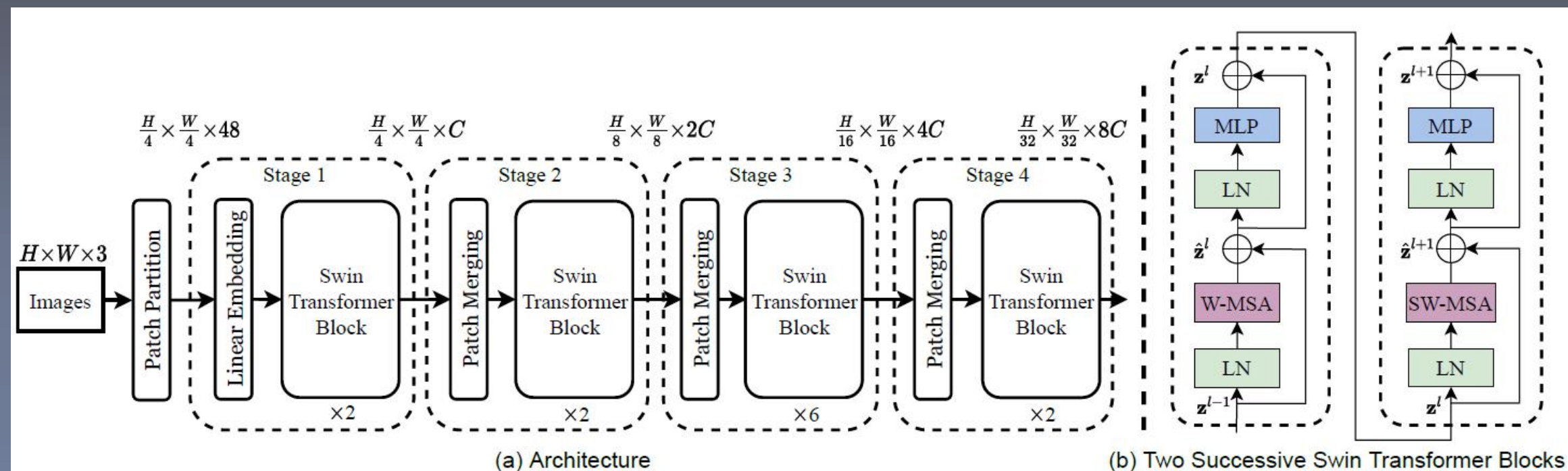
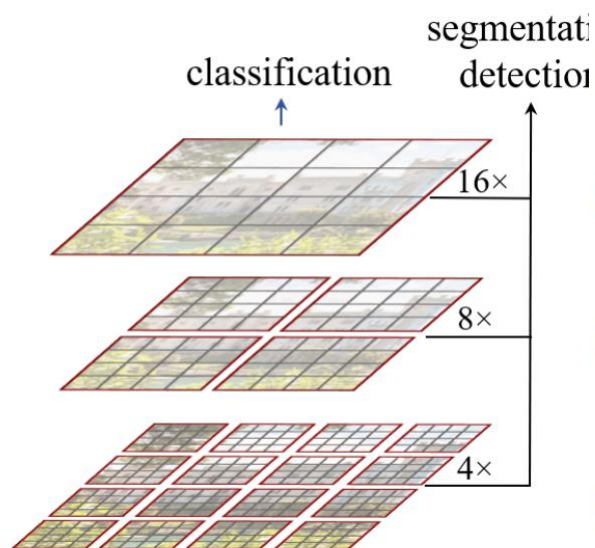


Figure 3. (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks (notation presented with Eq. (3)). W-MSA and SW-MSA are multi-head self attention modules with regular and shifted windowing configurations, respectively.

Swin Transformer 结构

Patch Merging

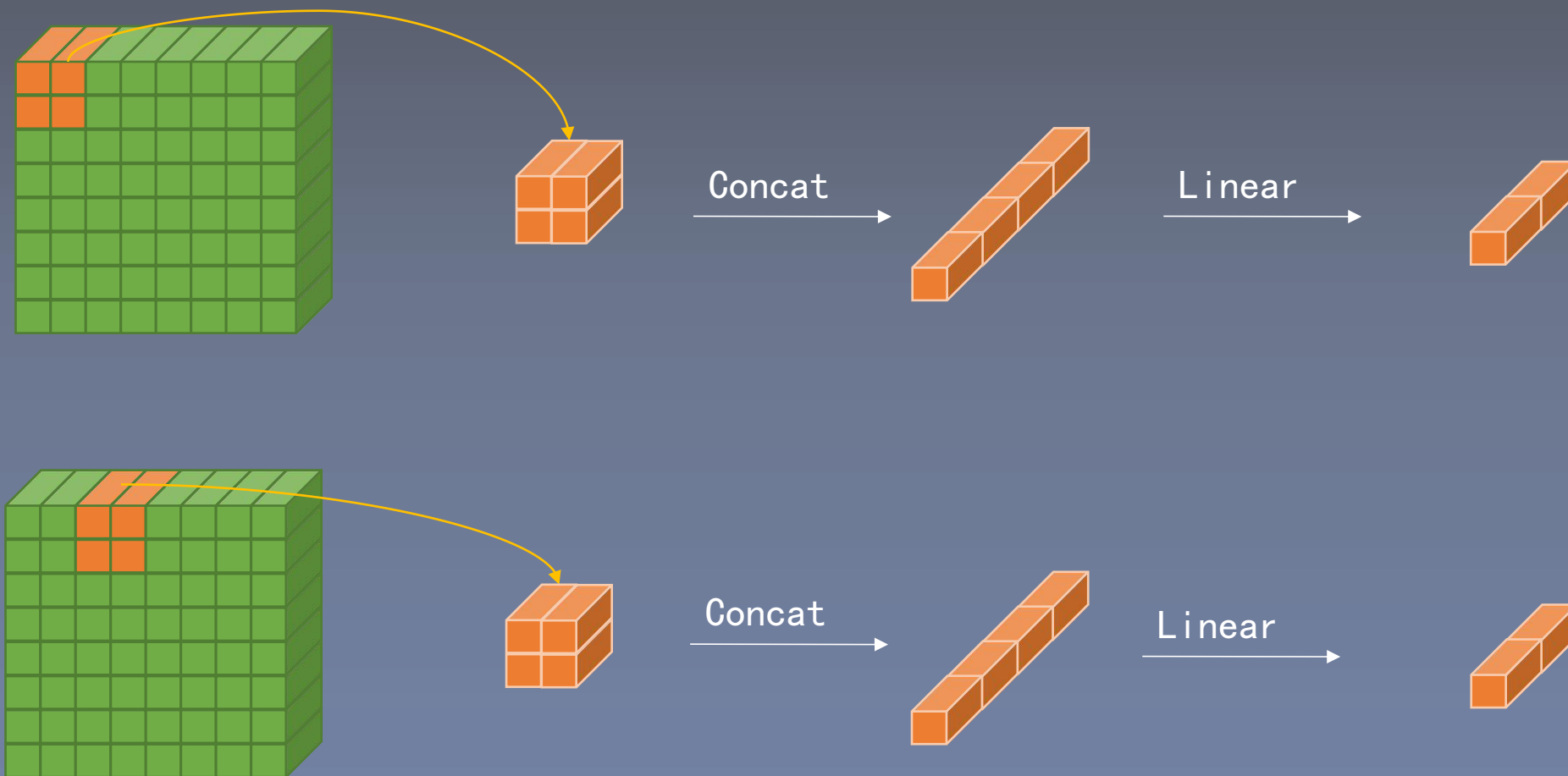


(a) Swin Transformer (ours)

To produce a hierarchical representation, the number of tokens is reduced by patch merging layers as the network gets deeper. The first patch merging layer concatenates the features of each group of 2×2 neighboring patches, and applies a linear layer on the $4C$ -dimensional concatenated features. This reduces the number of tokens by a multiple of $2 \times 2 = 4$ ($2 \times$ downsampling of resolution), and the output dimension is set to $2C$. Swin Transformer blocks are

Swin Transformer 结构

Patch Merging



互动

有没有什么疑问？

W-MSA

ViT上使用FPN的困境是什么？

MHA的困境：

q, k的数量和切分patch的大小相关

Patch越小, q, k越多, MHA计算越慢, 显存占用越高

Patch越大, feature Map分辨率越低, 这会影响到下游任务的性能

ViT上使用FPN的困境是什么？

Self Attention简短回顾

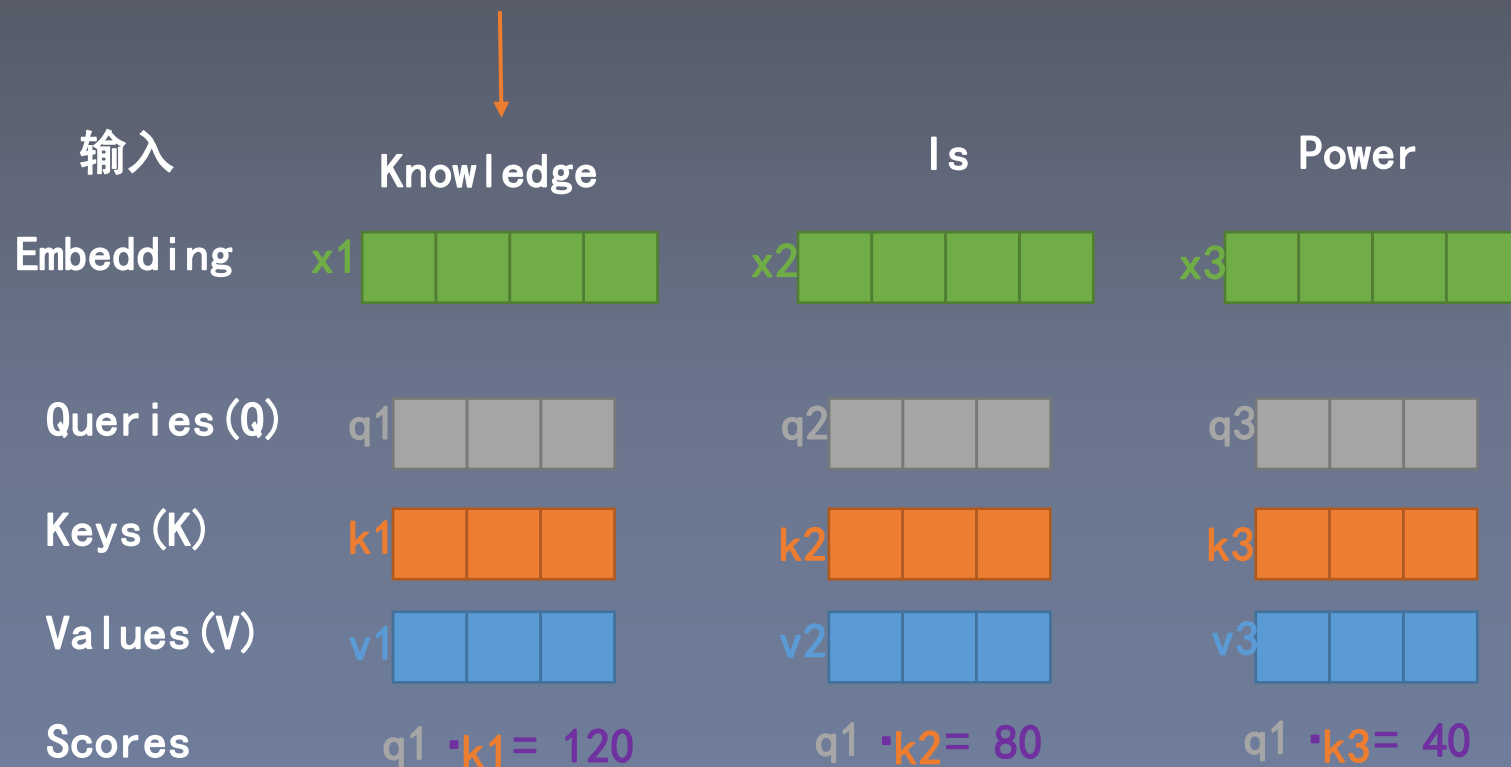
计算量上的开销：

需要进行pair(q, k)的相似度计算

所以时间复杂度是 $O(n^2)$

公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



PVT的解决方案

解决方案SRA: 人为的限制的K, V的个数

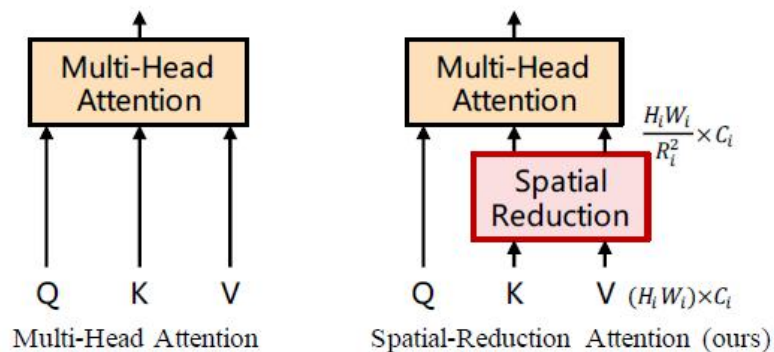


Figure 4: **Multi-head attention (MHA) vs. spatial-reduction attention (SRA).** With spatial-reduction operation, the computational/memory cost of our SRA could be much lower than that of MHA, so our SRA is more friendly to high-resolution feature maps.

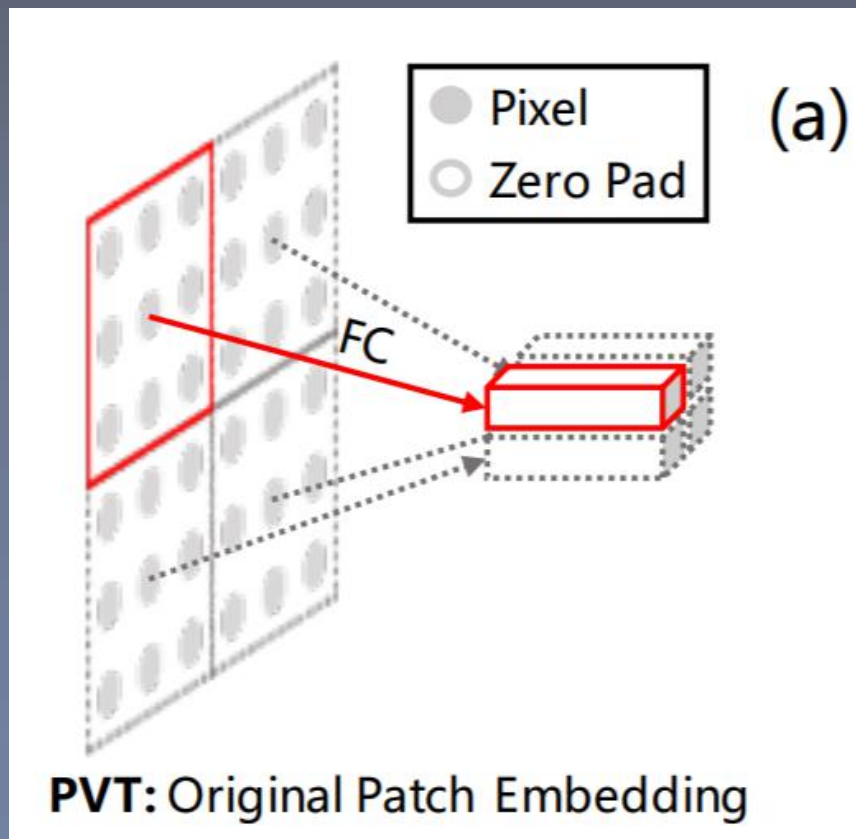
Similar to MHA, our SRA also receives a query Q , a key K , and a value V as input, and outputs a refined feature. The difference is that our SRA will reduce the spatial scale of K and V before the attention operation as shown in Figure 4, which largely reduces the computation/memory overhead. Details of SRA in the stage i can be formulated as follows:

$$\text{SRA}(Q, K, V) = \text{Concat}(\text{head}_0, \dots, \text{head}_{N_i})W^O, \quad (1)$$

$$\text{head}_j = \text{Attention}(QW_j^Q, \text{SR}(K)W_j^K, \text{SR}(V)W_j^V), \quad (2)$$

PVT的解决方案

在Feature Map上继续切分patch做出的



```
class Attention(nn.Module):
    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None, attn_drop=0., proj_drop=0., sr_ratio=1):
        super().__init__()
        assert dim % num_heads == 0, f"dim {dim} should be divided by num_heads {num_heads}."

        self.dim = dim
        self.num_heads = num_heads
        head_dim = dim // num_heads
        self.scale = qk_scale or head_dim ** -0.5

        self.q = nn.Linear(dim, dim, bias=qkv_bias)
        self.kv = nn.Linear(dim, dim * 2, bias=qkv_bias)
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop)

        self.sr_ratio = sr_ratio
        if sr_ratio > 1:
            self.sr = nn.Conv2d(dim, dim, kernel_size=sr_ratio, stride=sr_ratio)
            self.norm = nn.LayerNorm(dim)
```

SRA 的问题

Q与K的感受野不一样，而且复杂度还是 $O(n^2)$ ，只是限制了常数项。

在224，800等常用输入尺寸下总复杂度可以接受，但是继续放大会困难。

所以，能不能让QK的感受野一样的情况下，还把复杂度降下来？

W-MSA

Idea

既然问题是每来一个Q都需要和**所有的K**做计算

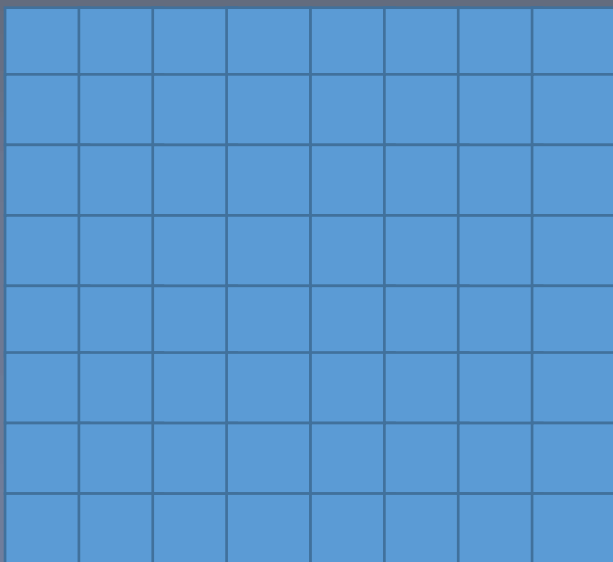
最简单直接的降低复杂度的方式就是Q只与**固定数量的K**做计算

那么是哪些K呢→空间**位置上接近**的那些K→弄个window好了

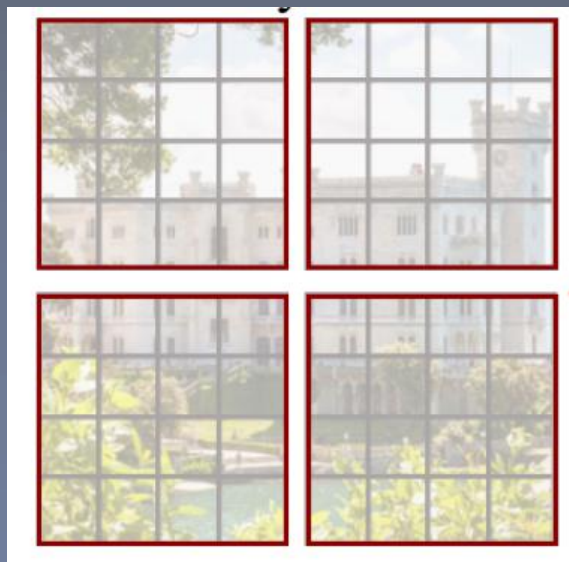
W-MSA

Idea详解

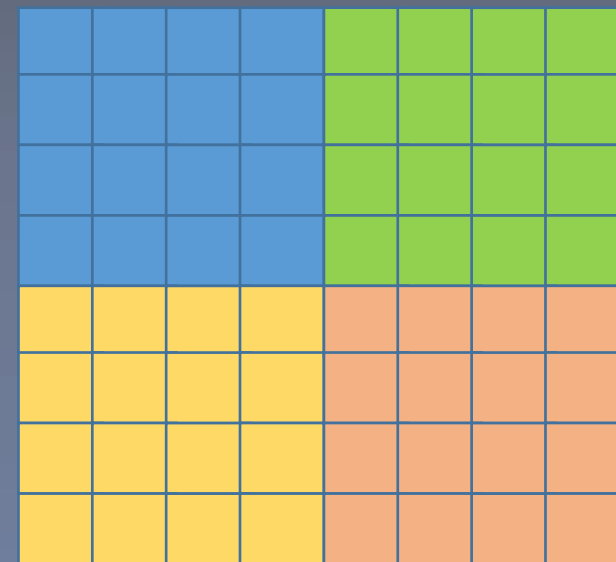
Swin Transformer使用window multiscale self attention, 将attention的计算限制在同一个窗口内, 使得复杂度降到了 $O(n)$



原本的计算量: 64×64



MSA限制在窗口中

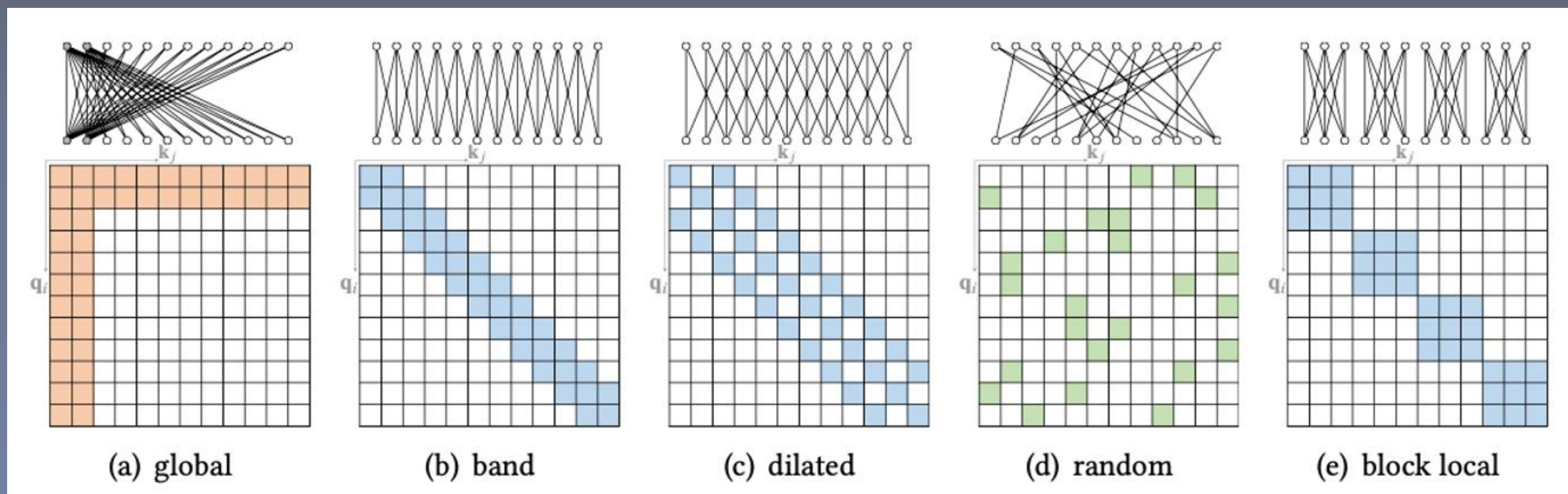


现在的计算量: $16 \times 16 \times 4$



W-MSA

Swin Transformer使用window multiscale self attention, 将attention的计算限制在同一个窗口内, 使得复杂度降到了 $O(n)$



- Link: <https://arxiv.org/pdf/2106.04554.pdf>

互动

有没有什么疑问？

SW-MSA



W-MSA的问题

显然这样硬性的限制会丢失全局信息，限制模型能力，因此需要一个跨Window的操作，增强window间的交互

Shifted window partitioning in successive blocks The window-based self-attention module lacks connections across windows, which limits its modeling power. To introduce cross-window connections while maintaining the efficient computation of non-overlapping windows, we propose a shifted window partitioning approach which alternates between two partitioning configurations in consecutive Swin Transformer blocks.

W-MSA的问题

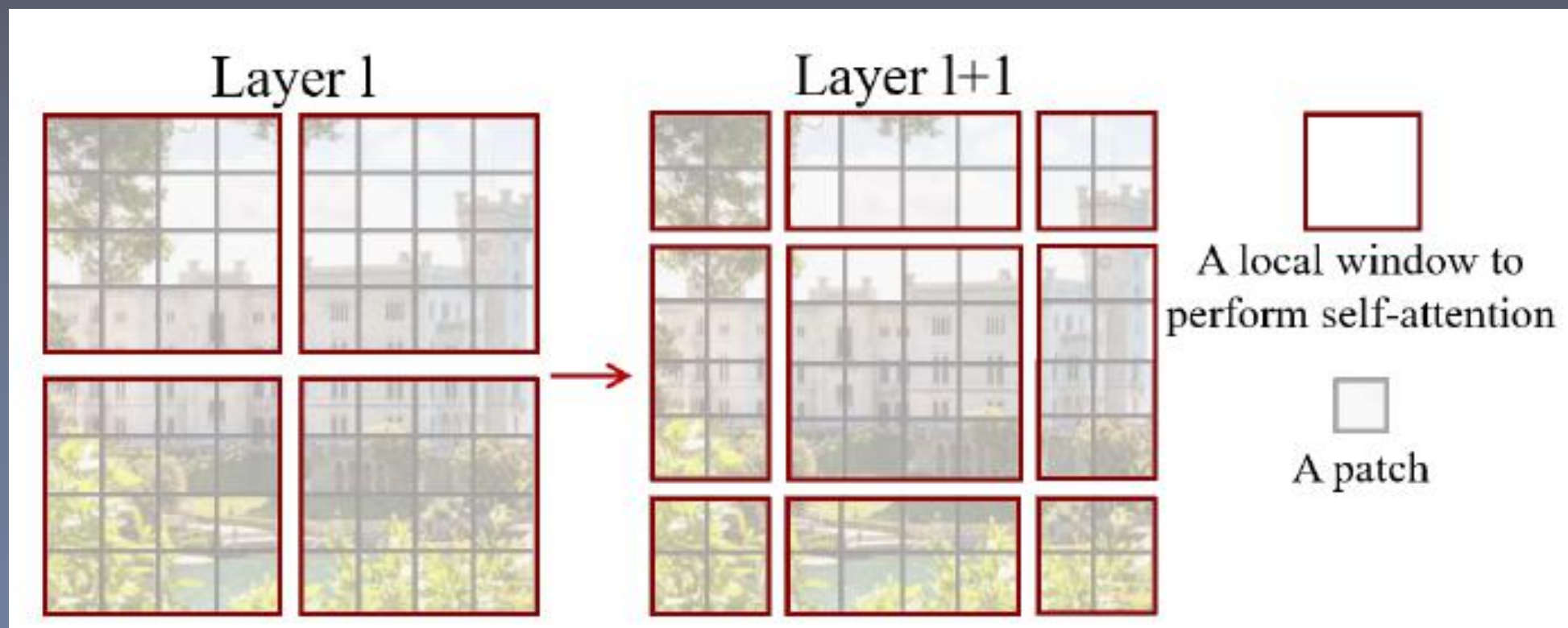
显然这样硬性的限制会丢失全局信息，限制模型能力，因此需要一个跨Window的操作，增强window间的交互



W-MSA



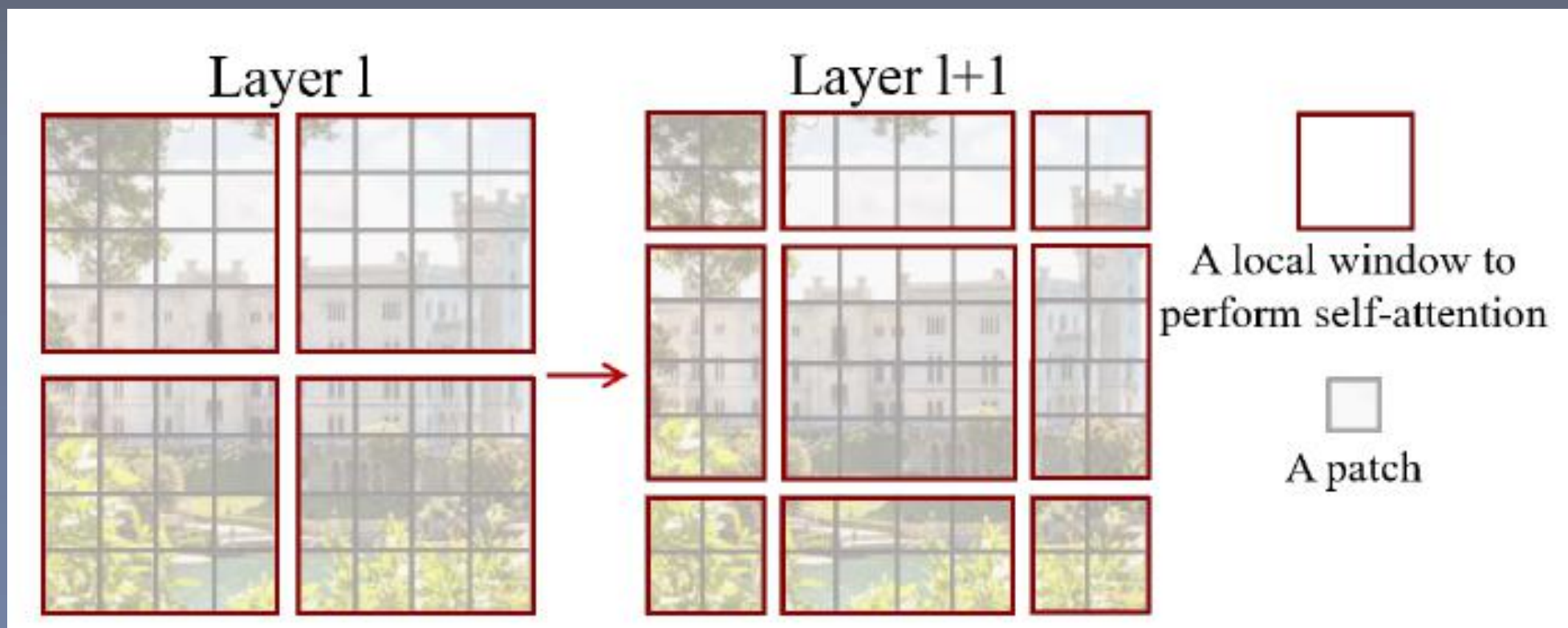
SW-MSA



SW-MSA

SW-MSA实现时候的问题:

移动切分后, window的数量变多了, 而且window的尺寸不一样了。

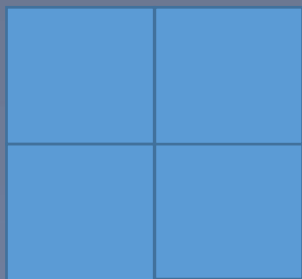


SW-MSA 的加速方法

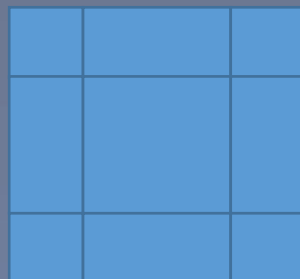
Naïve版本

可以通过padding的方式将window的尺寸变一样，在mask掉其他值，这样就可以直接调用原本的方法了

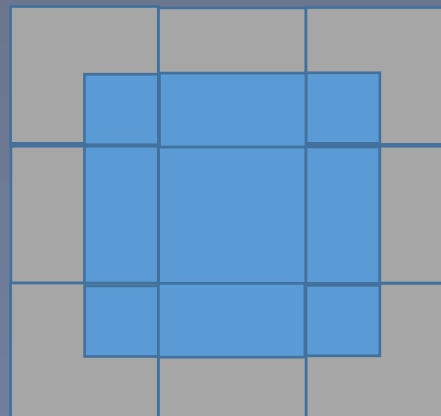
W-MSA



SW-MSA



Padding W-MSA

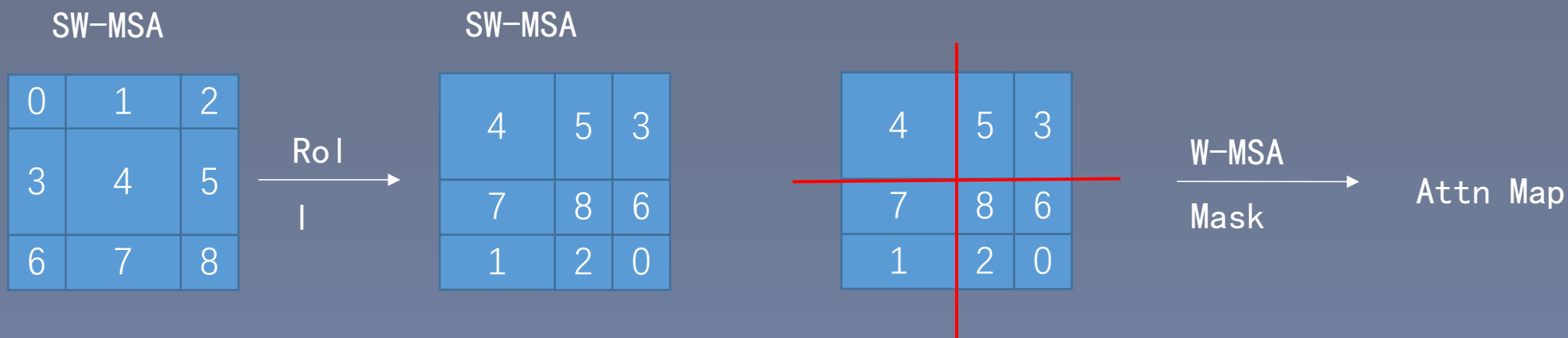


但这样计算量大增

SW-MSA 的加速方法

Cyclic-shift

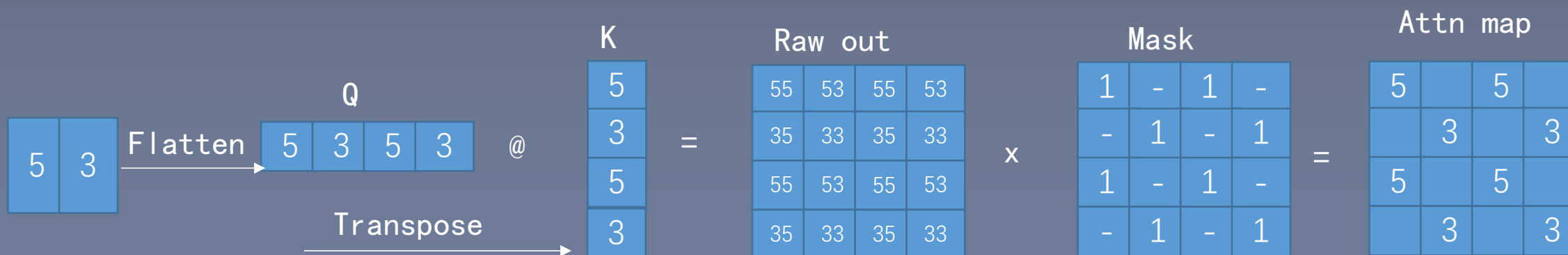
可以通过roll的方式将切碎的小window从左上角挪动到右下角，拼一起，
计算时候把不同window间的值mask掉，计算完成后再roll回来。



SW-MSA 的加速方法

Cyclic-shift

可以通过roll的方式将切碎的小window从左上角挪动到右下角，拼一起，
计算时候把不同window间的值mask掉，计算完成后再roll回来。



相对位置编码

相对位置编码

相对与绝对

- 绝对：对于每个位置生成一个固定的位置编码
- 相对：以不同位置为原点，其他位置的编码是不一样的
- 实现：维护一个Table，根据相对位置的偏移量，从Table中取编码值。

互动

- 具体细节我们去代码课配合图一行一行看

```
# define a parameter table of relative position bias
self.relative_position_bias_table = nn.Parameter(
    torch.zeros((2 * window_size[0] - 1) * (2 * window_size[1] - 1), num_heads)) # 2*Wh-1 * 2*Ww-1, nH

# get pair-wise relative position index for each token inside the window
coords_h = torch.arange(self.window_size[0])
coords_w = torch.arange(self.window_size[1])
coords = torch.stack(torch.meshgrid([coords_h, coords_w])) # 2, Wh, Ww
coords_flatten = torch.flatten(coords, 1) # 2, Wh*Ww
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :] # 2, Wh*Ww, Wh*Ww
relative_coords = relative_coords.permute(1, 2, 0).contiguous() # Wh*Ww, Wh*Ww, 2
relative_coords[:, :, 0] += self.window_size[0] - 1 # shift to start from 0
relative_coords[:, :, 1] += self.window_size[1] - 1
relative_coords[:, :, 0] *= 2 * self.window_size[1] - 1
relative_position_index = relative_coords.sum(-1) # Wh*Ww, Wh*Ww
self.register_buffer("relative_position_index", relative_position_index)
```

互动

有没有什么疑问？

CNN与Transformer的异同

CNN与Trans的异同

尽管他们看起来很像

- 操作都在window内
- 都是金字塔结构
- 都是Pretrain-Fine Turn的Pipeline
- 甚至只要你愿意可以用Conv2d写一个Trans出来
- 但他们在处理图像时候的**逻辑**是完全不一样的, CNN有很强的**Inductive Bias**, Trans没有

CNN与Trans的异同

Inductive Bias

- 直译为归纳偏置
- 归纳→总结，也就是从现存的例子中找到一些比较通用的规则
- 偏置→选择偏好。
- 整理在一起可以理解为：在面对一些特定问题的时候，我们认为模型应该会有哪些特点会比较容易work，因此而**做出的一系列对模型的人为限制**。
- 比如图像处理中，每个位置的信息与周围的信息相关，因此设计出conv。
- 比如NLP中，认为输出的结果与单词的先后顺序相关，因此设计出RNN。

CNN与Trans的异同

CNN的inductive bias

CNNs的假设及奇怪性质

- Pixel只与周围Pixel相关→局部连接→Conv固定大小
- 空间平移不变性 → 权重共享
- 为什么常见的conv都是奇数大小的？
- 为什么深层的Feature Map难以理解？
- 为什么从ImageNet迁移到肿瘤分类仍然有不错的性能？

CNN与Trans的异同

A swift and brutal intro to CNNs

从滤波器说起

右侧是一个边缘滤波器

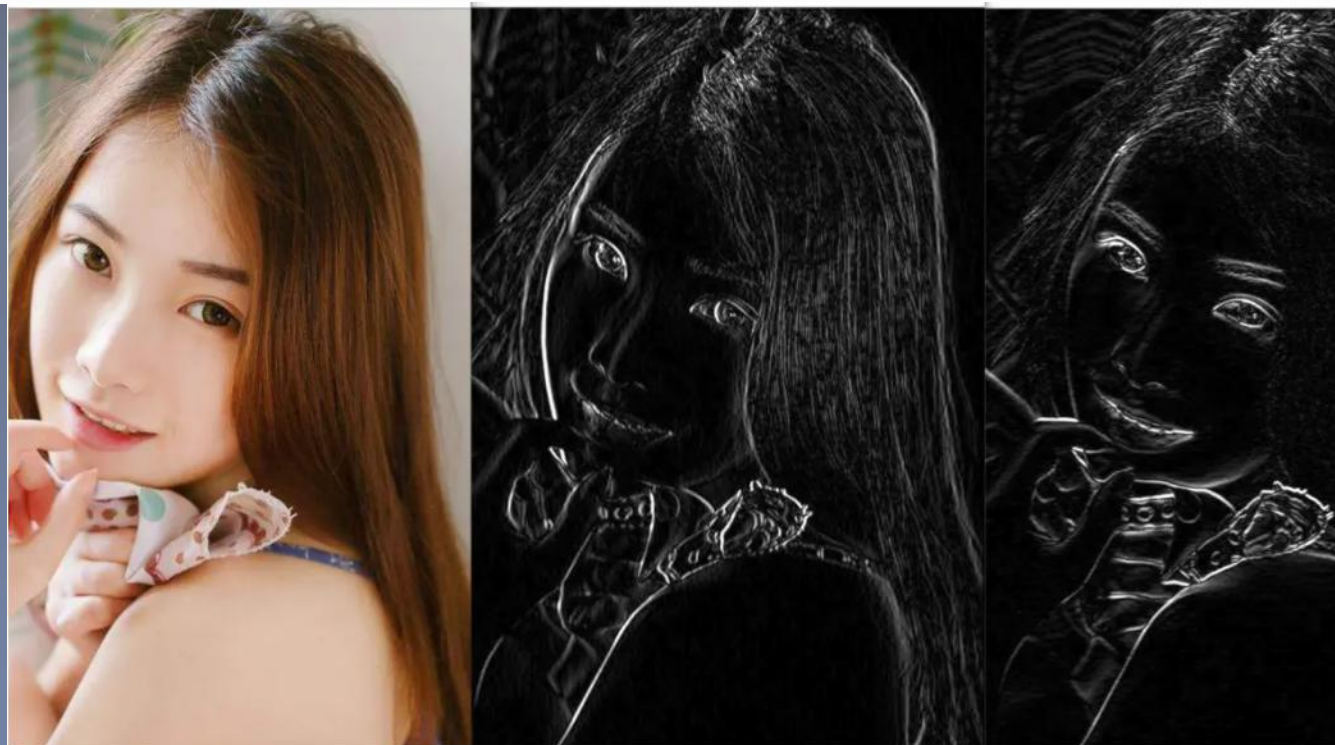
Kernel内不同的数值会对不同的输入产生不同的激活值

边缘滤波器中边缘的位置激活值高
角点滤波器中角点位置的激活值高

而CNN中kernel内的数值可以学习就达到了特征提取的目的。

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$





CNN与Trans的异同

A swift and brutal intro to CNNs

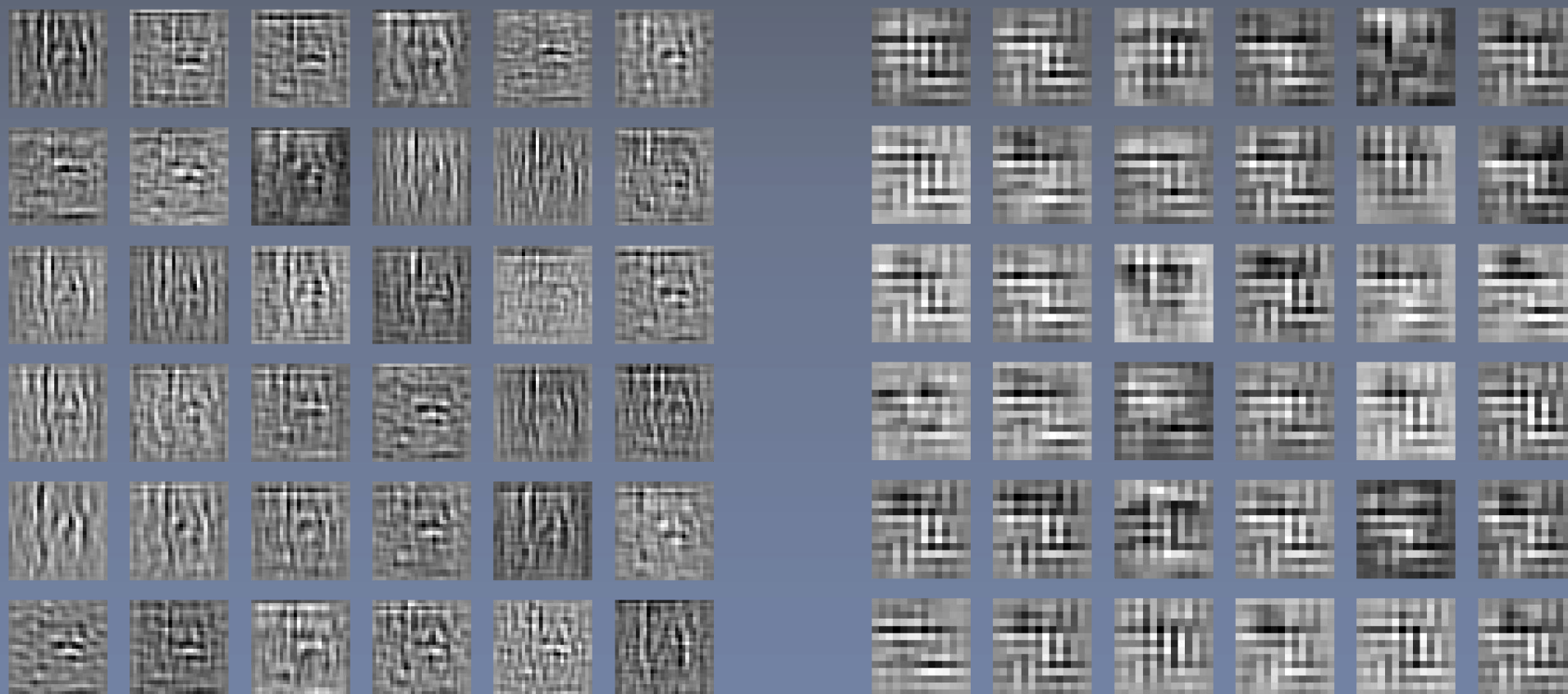
R50 Feature Map可视化



CNN与Trans的异同

A swift and brutal intro to CNNs

R50 Feature Map可视化

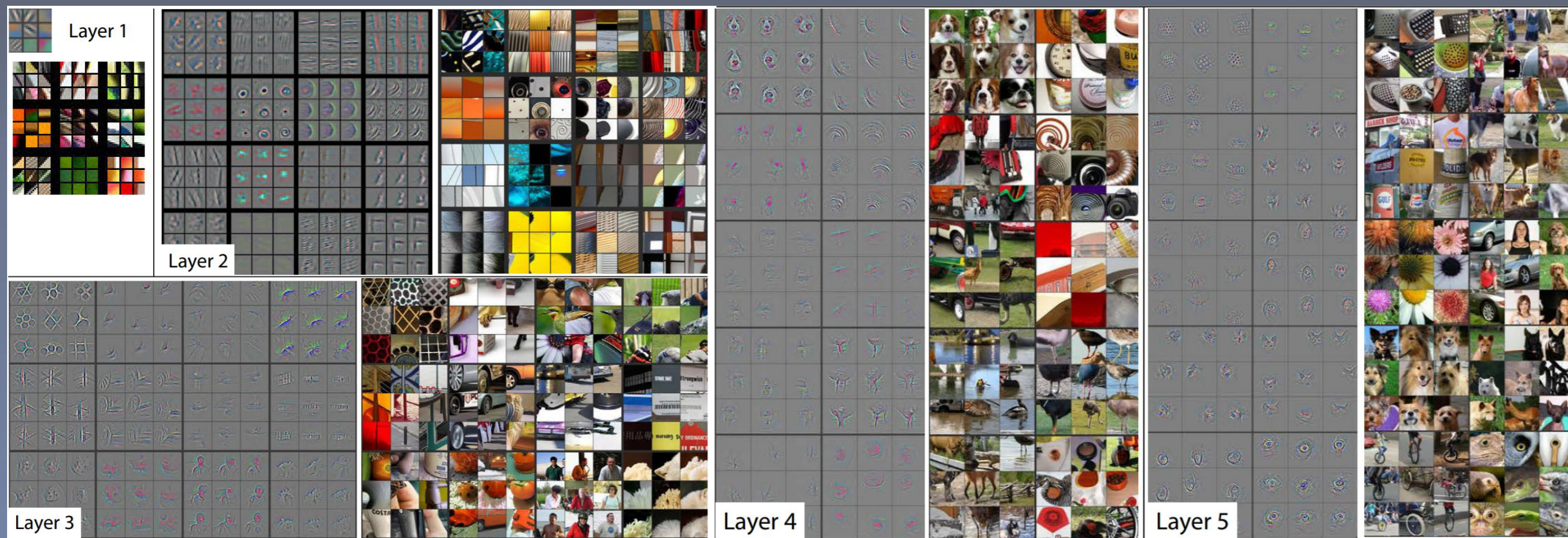




CNN与Trans的异同

A swift and brutal intro to CNNs

Visualizing and Understanding Convolutional Networks



CNN与Trans的异同

A swift and brutal intro to CNNs

空间平移不变性→权重共享



CNN与Trans的异同

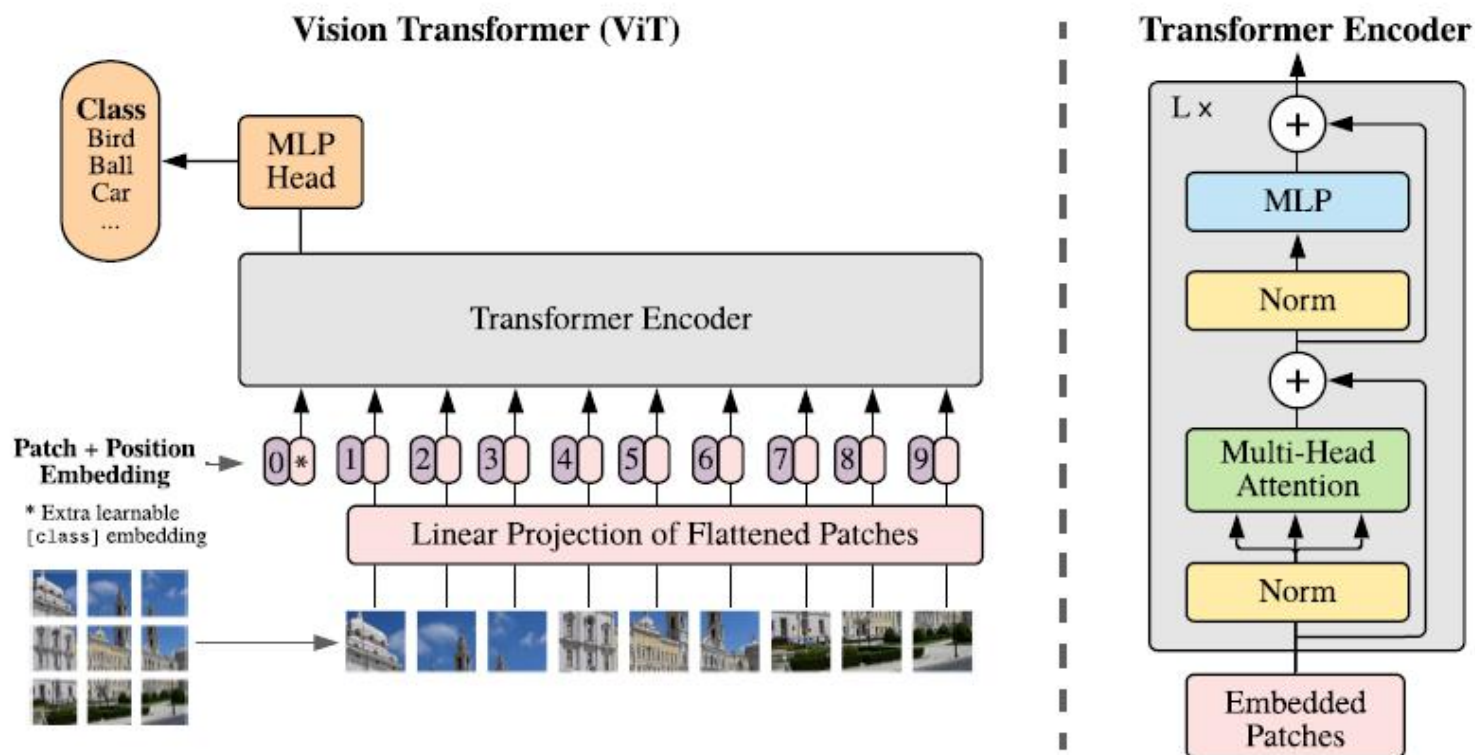
ViT总结

ViT把事情拆开来做

Patch内信息建模→MLP

Patch间信息建模→MHA

这之中的限定相对Conv就要小
Inductive bias就小



CNN与Trans的异同

ViT总结

MLP Mixer

Patch内信息建模→MLP
Patch间信息建模→MLP

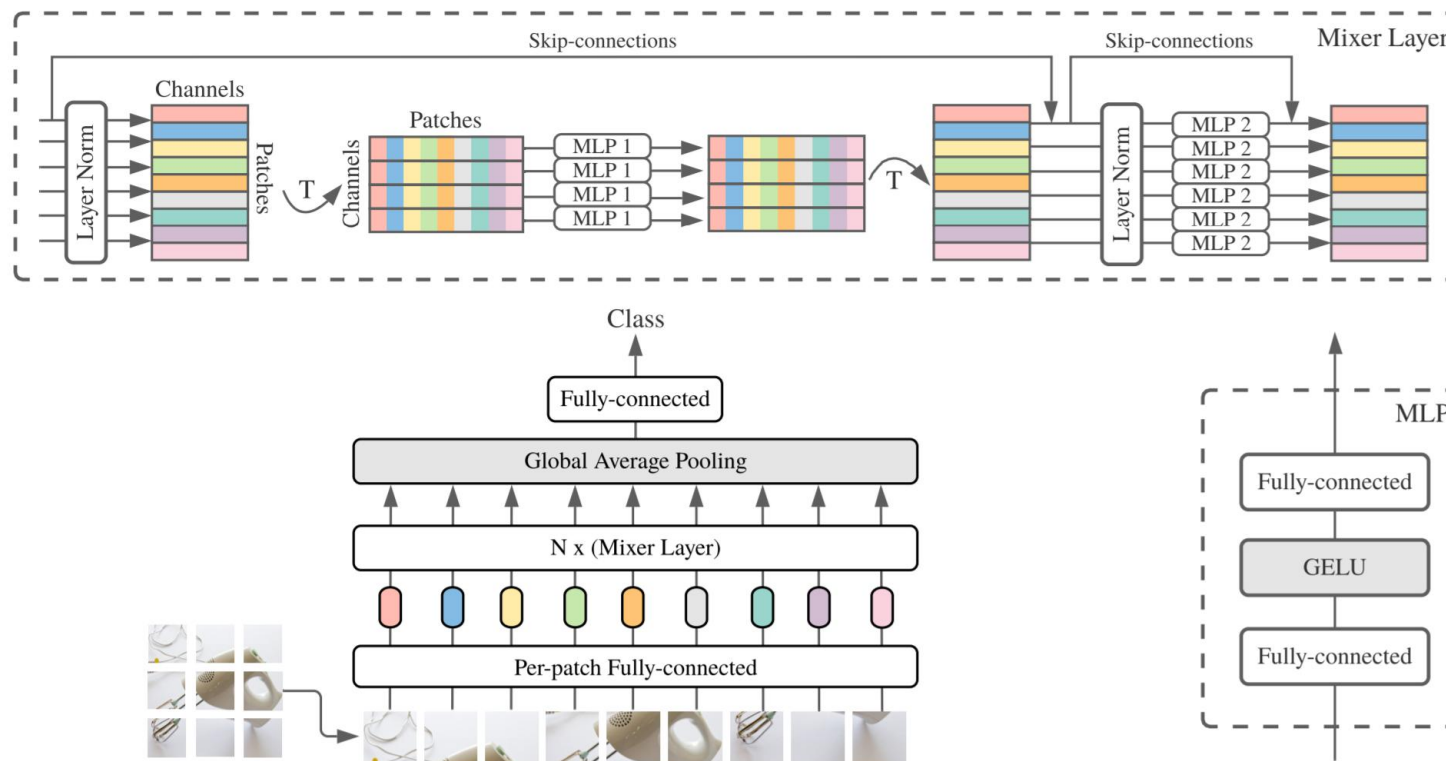


Figure 1: MLP-Mixer consists of per-patch linear embeddings, Mixer layers, and a classifier head. Mixer layers contain one token-mixing MLP and one channel-mixing MLP, each consisting of two fully-connected layers and a GELU nonlinearity. Other components include: skip-connections, dropout, and layer norm on the channels.

互动

有没有什么疑问？

实验设置及结果

实验设置及结果

Experiments

Image Classification: 设置上 follow DeiT

- Input size : 224 x 224
- Batch size : 128
- Learning Rate : 0.001
- Total Epoch : 300
- LR Schedule : cosine
- Resource : 8 V100 GPUs

- *Regular ImageNet-1K training.* This setting mostly follows [60]. We employ an AdamW [36] optimizer for 300 epochs using a cosine decay learning rate scheduler and 20 epochs of linear warm-up. A batch size of 1024, an initial learning rate of 0.001, and a weight decay of 0.05 are used. We include most of the augmentation and regularization strategies of [60] in training, except for repeated augmentation [30] and EMA [44], which do not enhance performance. Note that this is contrary to [60] where repeated augmentation is crucial to stabilize the training of ViT.



实验设置及结果

Experiments

Image Classification上的实验结果:

- 可以看到Swin在参数量和GFLOPs与baseline差不多的情况下会有很大的性能提升。

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2



实验设置及结果

Experiments

Object Detection上的实验结果:

- 可以看到swin在各种setting上都涨点明显。

(a) Various frameworks							
Method	Backbone	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

(b) Various backbones w. Cascade Mask R-CNN								
	AP ^{box}	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask}	AP ^{mask} ₅₀	AP ^{mask} ₇₅	#param	FLOPsFPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G 10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G 18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G 15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G 12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G 12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G 10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G 11.6



实验设置及结果

Experiments

Semantic Segmentation上的实验结果:

- 可以看到把backbone从ResNet系列替换到swin系列, 会有稳定的性能提升。

ADE20K		val	test	#param.	FLOPs	FPS
Method	Backbone	mIoU	score			
DANet [22]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [10]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [23]	ResNet-101	45.9	38.5	-	-	-
DNL [68]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [70]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [66]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [70]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [10]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [10]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [78]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2



Ablation Studies



消融实验

Ablation Studies

Shifted windows:

- 带shifted windows比不带强

	ImageNet		COCO		ADE20k mIoU
	top-1	top-5	AP ^{box}	AP ^{mask}	
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1





消融实验

Ablation Studies

相对位置编码：

- no pos. 无位置编码
- abs. pos. ViT的位置编码
- abs + rel. pos. 两种编码相加
- app. 不除根号dk

	ImageNet		COCO		ADE20k mIoU
	top-1	top-5	AP ^{box}	AP ^{mask}	
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1

Table 4. Ablation study on the *shifted windows* approach and different position embedding methods on three benchmarks, using the Swin-T architecture. w/o shifting: all self-attention modules adopt regular window partitioning, without *shifting*; abs. pos.: absolute position embedding term of ViT; rel. pos.: the default settings with an additional relative position bias term (see Eq. (4)); app.: the first scaled dot-product term in Eq. (4).





消融实验

Ablation Studies

不同的self attention方法:

method	MSA in a stage (ms)				Arch. (FPS)		
	S1	S2	S3	S4	T	S	B
sliding window (naive)	122.5	38.3	12.1	7.6	183	109	77
sliding window (kernel)	7.6	4.7	2.7	1.8	488	283	187
Performer [13]	4.8	2.8	1.8	1.5	638	370	241
window (w/o shifting)	2.8	1.7	1.2	0.9	770	444	280
shifted window (padding)	3.3	2.3	1.9	2.2	670	371	236
shifted window (cyclic)	3.0	1.9	1.3	1.0	755	437	278

Table 5. Real speed of different self-attention computation methods and implementations on a V100 GPU.

	Backbone	ImageNet		COCO		ADE20k mIoU
		top-1	top-5	AP ^{box}	AP ^{mask}	
sliding window	Swin-T	81.4	95.6	50.2	43.5	45.8
Performer [13]	Swin-T	79.0	94.2	-	-	-
shifted window	Swin-T	81.3	95.6	50.5	43.7	46.1

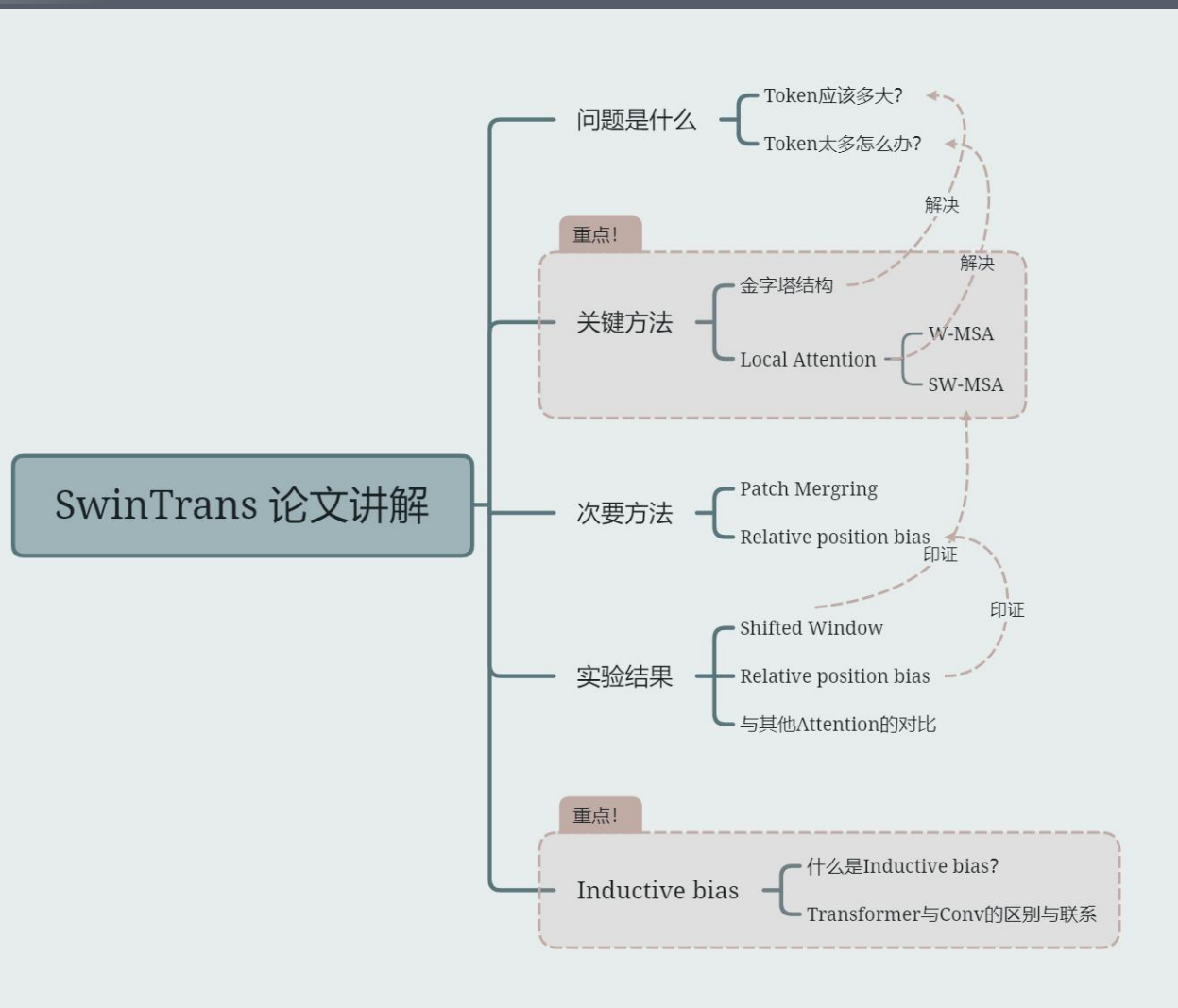
Table 6. Accuracy of Swin Transformer using different methods for self-attention computation on three benchmarks.



论文总结

论文总结

Summary of the paper



论文总结

Summary of the paper

A

关键点

- Swin Transformer的结构
- MHA的计算瓶颈问题
- W-MSA和SW-MSA都是如何操作的
- CNN的inductive bias

B

创新点

- 一种可以用顺畅用于下游任务的ViT
- Window内, local的attention大幅度降低了计算量。

讨论

Discussion



问题

- W-MSA有什么问题？
- 在此基础上有没有有什么创新的思路？

讨论解答

Discussion



解答

- W-MSA哪怕带Shift能够交互的Token还是少， 如何增加？
- 是否有其他降低Q， K数目的方法？

——我说——

在其他任何地方的投资，都没有投资自己回报率高，你是赚不到超出你认知以外的钱的

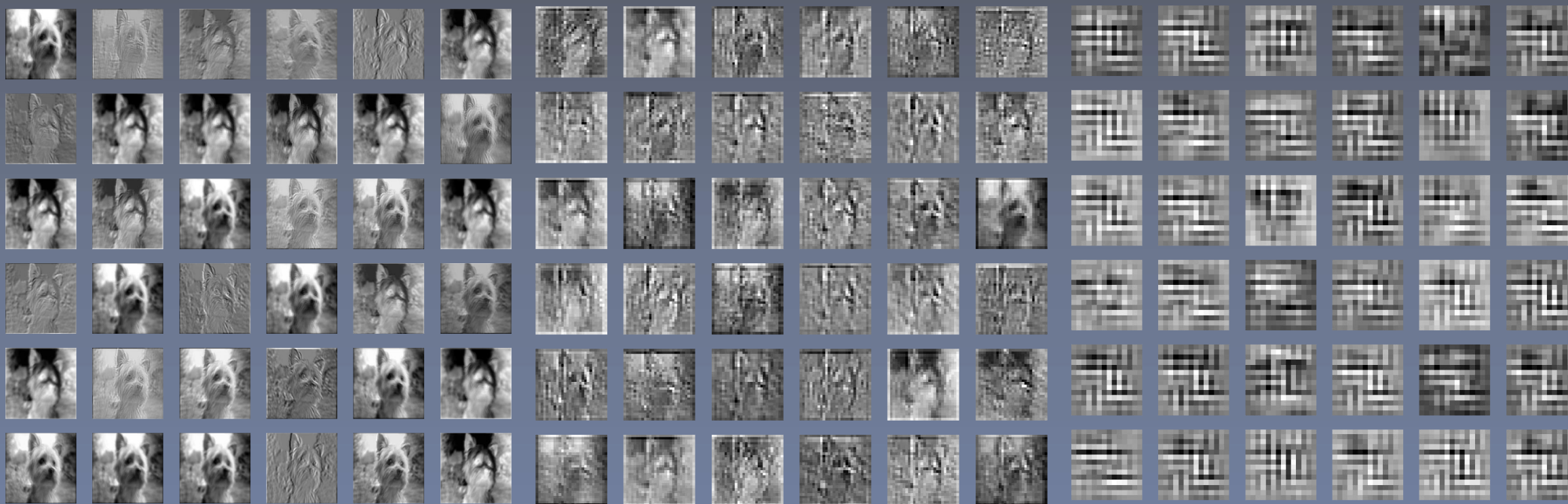
愿你成为一名终身学习者。

坚持不住的时候就再坚持一下，因为优秀并不是一个结果，
而是一种习惯



可视化

ResNet (local pyramid)

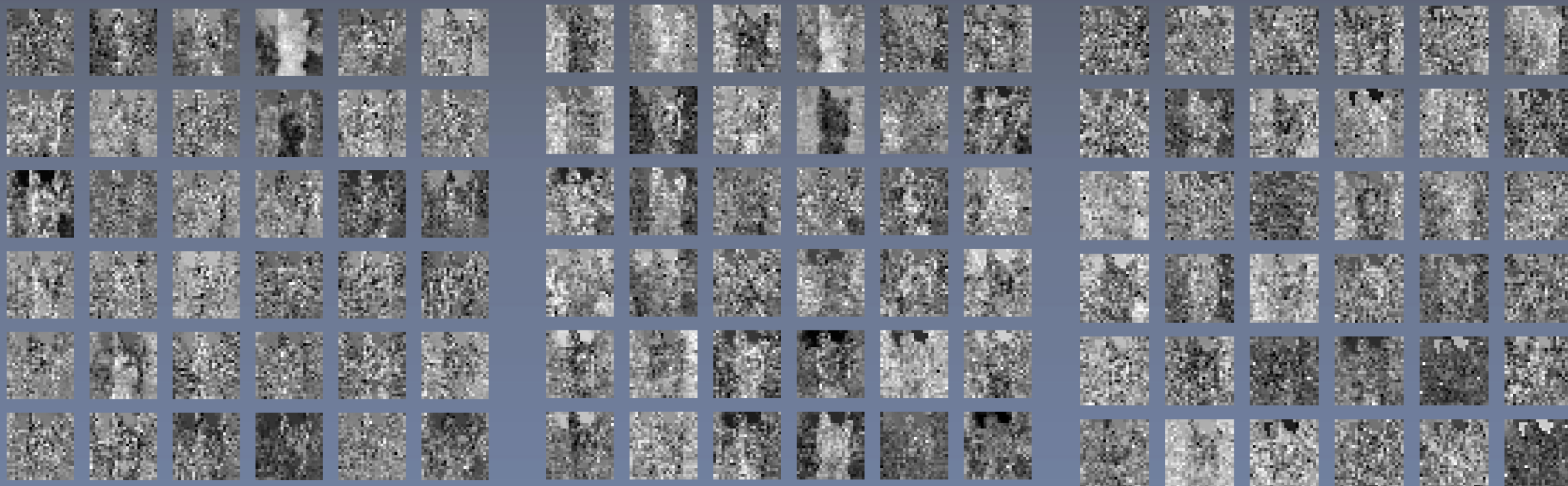


可视化

ViT (global col)

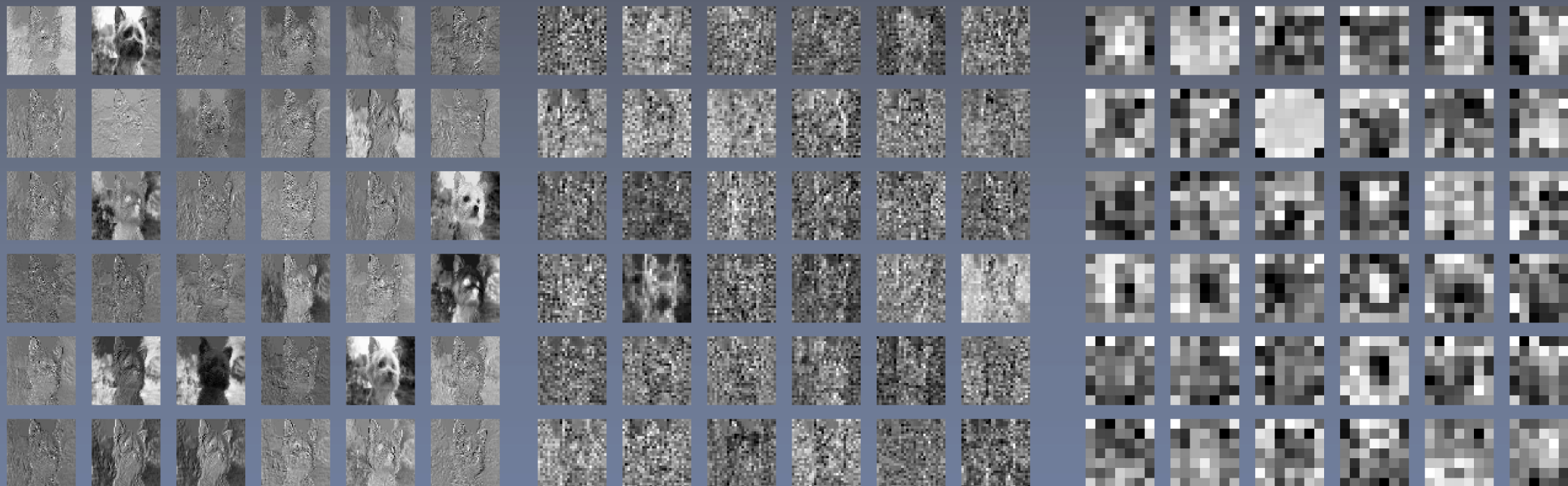


深度之眼
deepshare.net



可视化

Swin trans (local pyramid)



相对位置编码

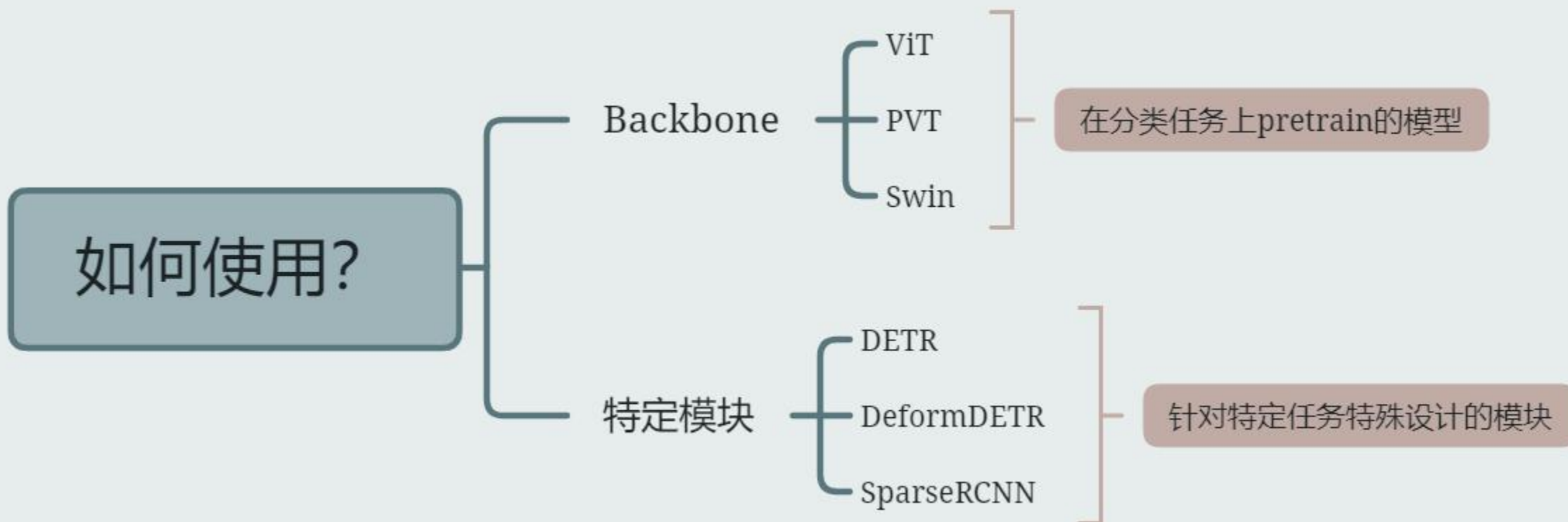
Swin trans (local pyramid)

0	1
2	3

	k0	k1	k2	k3
q0	0,0	0,-1	-1,0	-1,-1
q1	0,1	0,0	-1,1	-1,0
q2	1,0	1,-1	0,0	0,-1
q3	1,1	1,0	0,1	0,0

	k0	k1	k2	k3
q0	1,1	1,0	0,1	0,0
q1	1,2	1,1	0,2	0,1
q2	2,1	2,0	1,1	1,0
q3	2,2	2,1	1,2	1,1

如何在下游中使用Transformer？



如何在下游中使用Transformer？

什么是backbone？

- 进行特征提取的部分。
- 一般直接挪用分类网络，比如ResNet，VGG等。

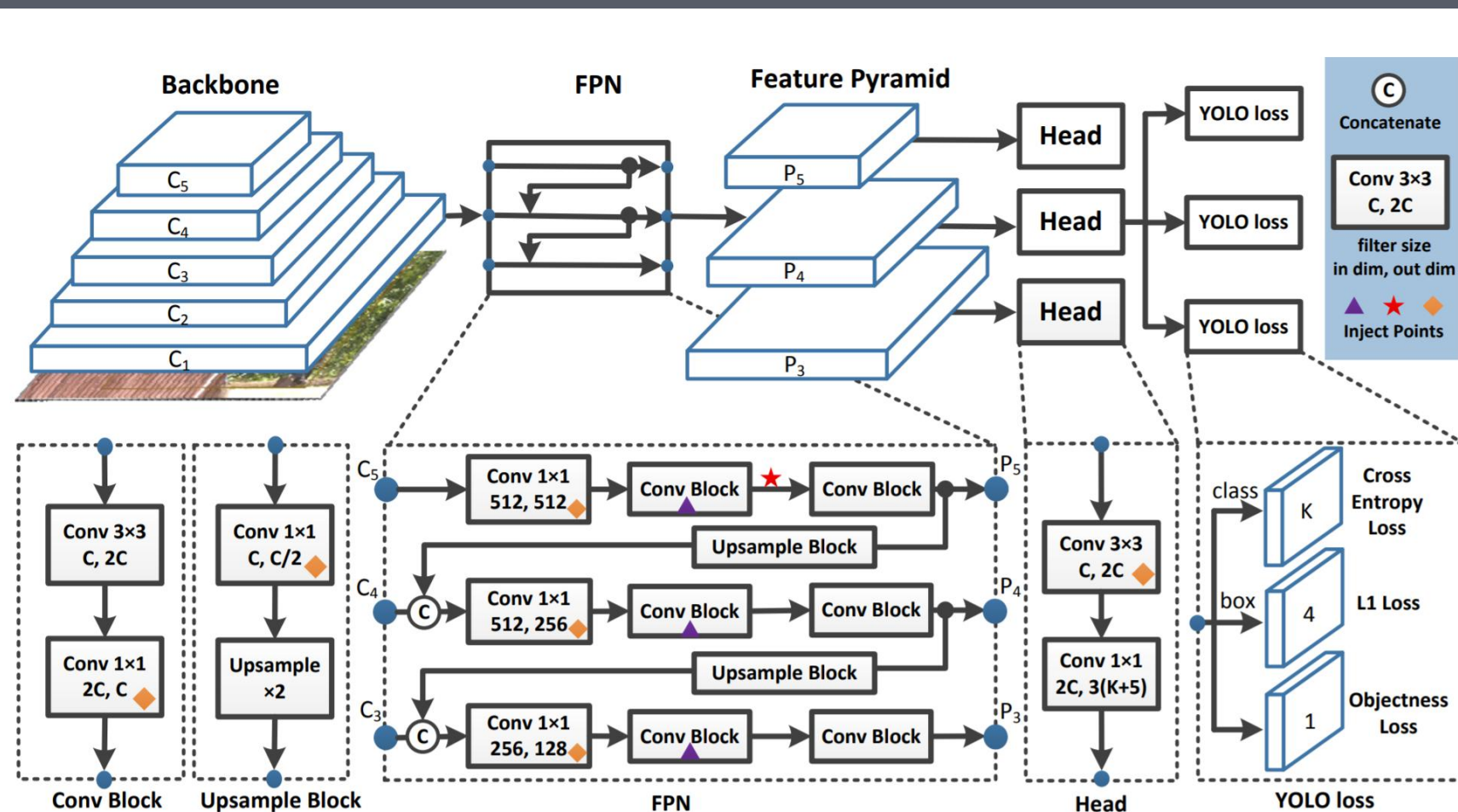


Figure 2. The network architecture of YOLOv3 and inject points for PP-YOLO. Activation layers are omitted for brevity. Details are described in Section 3.1 and Section 3.2.

如何在下游中使用Transformer？

什么是backbone？

- 进行特征提取的部分。
- 一般直接挪用分类网络，比如ResNet，VGG等。

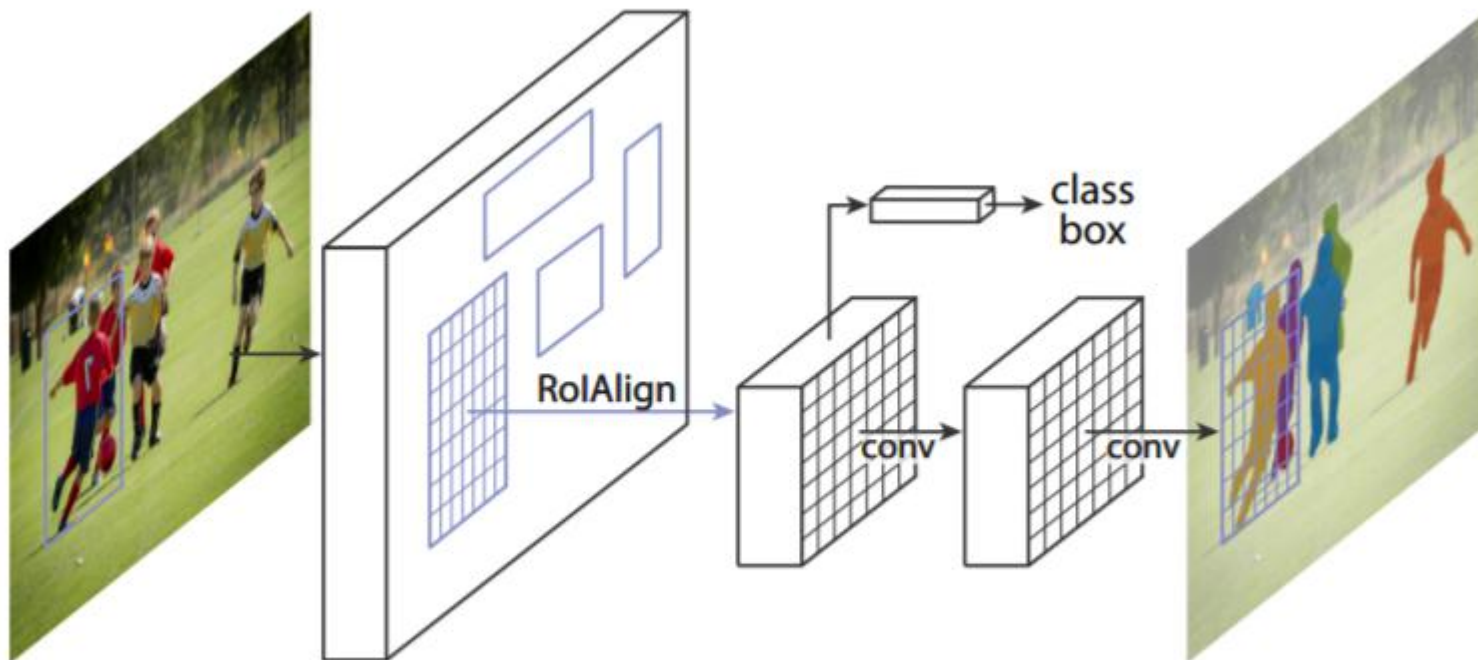


Figure 1. The **Mask R-CNN** framework for instance segmentation.

如何在下游中使用Transformer？

什么是backbone？

- 进行特征提取的部分。
- 一般直接挪用分类网络，比如ResNet，VGG等。

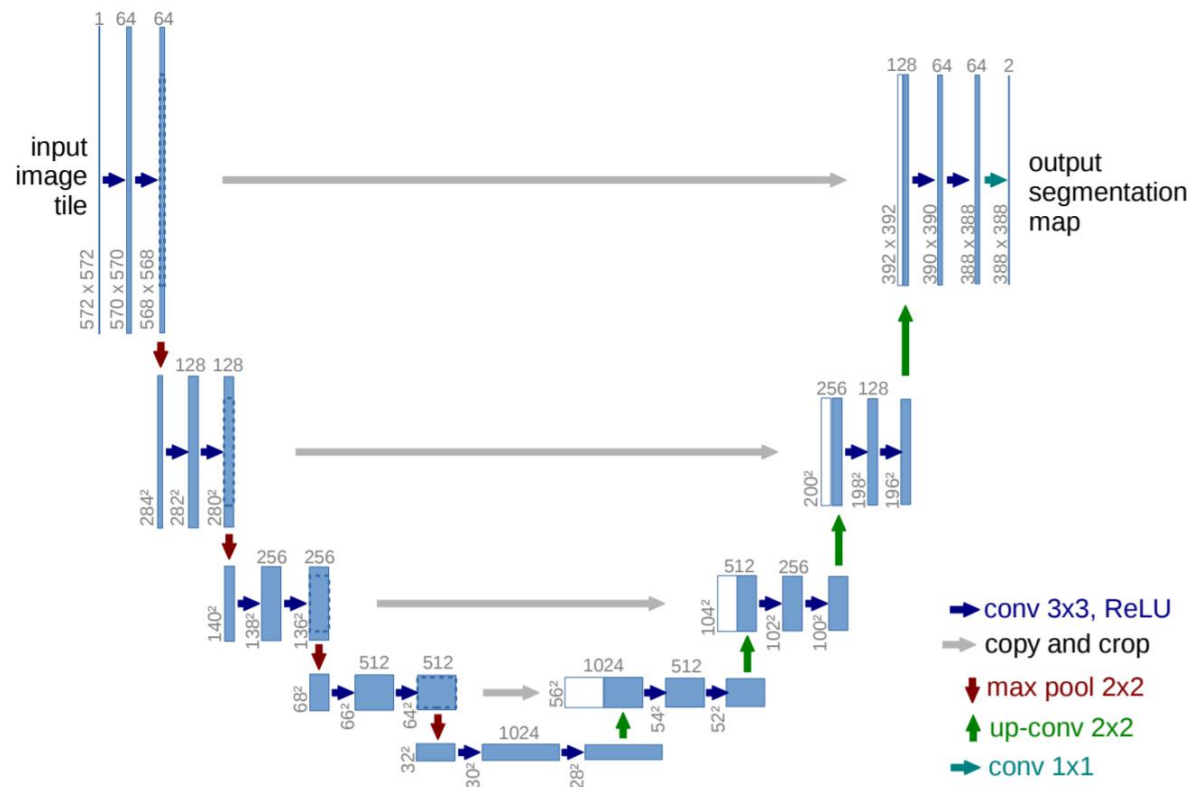


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

如何在下游中使用Transformer？

如何用？

- 1. 在代码中用swin 替换原本的r50
- 2. 下载训练好的权重
- 3. load权重开始训练



深度之眼
deepshare.net
